

Лабораторна робота №6.

Обробка аналогових даних у багатозадачній системі

Мета роботи. Реалізувати багатозадачну систему, у якій одна задача зчитує аналогові дані з потенціометра або фоторезистора, а інша – відображає їх на OLED-дисплеї.

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Завантажте в каталог вашого проєкту бібліотеки для роботи з OLED дисплеєм перейшовши за посиланням:

<https://github.com/mpavlyk/embedded-systems/blob/main/lib/c-sdk/lib.zip>

3. Відкрийте файл **CMakeLists.txt**, який знаходиться у каталозі вашого проєкту і відредагуйте його наступним чином:

- знайдіть рядок із інструкцією `pico_sdk_init()`. Після цієї інструкції вставте:

1	# Add Subdirectory
2	add_subdirectory(/lib/Config)
3	add_subdirectory(/lib/OLED)
4	add_subdirectory(/lib/Fonts)
5	add_subdirectory(/lib/GUI)
6	
7	# Include Directories
8	include_directories(/lib/Config)
9	include_directories(/lib/OLED)
10	include_directories(/lib/Fonts)
11	include_directories(/lib/GUI)

- Знайдіть інструкцію - `target_link_libraries(lab01 pico_stdlib)` і додайте бібліотеки - `hardware_spi`, `hardware_i2c`, `hardware_pwm`, `hardware_adc`, `Config`, `GUI`, `Fonts`, `OLED`.

1	target_link_libraries(lab-1
2	pico_stdlib
3	FreeRTOS-Kernel
4	FreeRTOS-Kernel-Heap4
5	hardware_spi
6	hardware_i2c
7	hardware_pwm

8	hardware_adc
9	Config
10	GUI
11	Fonts
12	OLED)

4. Скомпілюйте та завантажте наведений код у Pico. В даному коді наведений приклад виведення інформації на дисплей.

```

1  #include <stdlib.h>
2  #include "pico/stdlib.h"
3  #include "DEV_Config.h"
4  #include "GUI_Paint.h"
5  #include "OLED_1in5.h"
6  #include "Debug.h"
7
8  int main(void)
9  {
10     DEV_Delay_ms(100);
11
12
13     if(DEV_Module_Init()!=0){
14         while(1){
15             printf("END\r\n");
16         }
17     }
18     printf("OELD_test Demo\r\n");
19     /* Init */
20     OLED_1in5_Init();
21     OLED_1in5_Clear(0);
22
23     UBYTE *BlackImage;
24     UWORD Imagesize = ((OLED_1in5_WIDTH%2==0)? (OLED_1in5_WIDTH/2):
25 (OLED_1in5_WIDTH/2+1)) * OLED_1in5_HEIGHT;
26     if((BlackImage = (UBYTE *)malloc(Imagesize)) == NULL) {
27         while(1){
28             printf("Failed to apply for black memory...\r\n");
29         }
30     }
31     printf("Paint_NewImage\r\n");
32     Paint_NewImage(BlackImage, OLED_1in5_WIDTH, OLED_1in5_HEIGHT, 0,
33 WHITE);
34     Paint_SetScale(16);
35
36     printf("Drawing\r\n");

```

```

//1.Select Image
Paint_SelectImage(BlackImage);
DEV_Delay_ms(500);
Paint_Clear(BLACK);

// 2.Drawing on the image
printf("Drawing:page 1\r\n");
Paint_DrawPoint(20, 10, WHITE, DOT_PIXEL_1X1, DOT_STYLE_DFT);
Paint_DrawPoint(30, 10, WHITE, DOT_PIXEL_2X2, DOT_STYLE_DFT);
Paint_DrawPoint(40, 10, WHITE, DOT_PIXEL_3X3, DOT_STYLE_DFT);
Paint_DrawLine(10, 10, 10, 20, WHITE, DOT_PIXEL_1X1,
LINE_STYLE_SOLID);
Paint_DrawLine(20, 20, 20, 30, WHITE, DOT_PIXEL_1X1,
LINE_STYLE_SOLID);
Paint_DrawLine(30, 30, 30, 40, WHITE, DOT_PIXEL_1X1,
LINE_STYLE_DOTTED);
Paint_DrawLine(40, 40, 40, 50, WHITE, DOT_PIXEL_1X1,
LINE_STYLE_DOTTED);
Paint_DrawCircle(60, 30, 15, WHITE, DOT_PIXEL_1X1,
DRAW_FILL_EMPTY);
Paint_DrawCircle(100, 40, 20, WHITE, DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(50, 30, 60, 40, WHITE, DOT_PIXEL_1X1,
DRAW_FILL_EMPTY);
Paint_DrawRectangle(90, 30, 110, 50, BLACK, DOT_PIXEL_1X1,
DRAW_FILL_FULL);
// 3. Show image on page1

OLED_1in5_Display(BlackImage);
DEV_Delay_ms(10000);
Paint_Clear(BLACK);
Paint_DrawString_EN(5,10,"128 x 128 Pixels", &Font12, WHITE, BLACK);
Paint_DrawString_EN(5,25,"OLED-1.5-Demo", &Font12, WHITE, BLACK);
Paint_DrawString_EN(5,40,"Waveshare", &Font12, WHITE, BLACK);
Paint_DrawRectangle(0, 55 , OLED_1in5_WIDTH-1, 65 , 0x01,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 65 , OLED_1in5_WIDTH-1, 75 , 0x02,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 75 , OLED_1in5_WIDTH-1, 85 , 0x04,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 85 , OLED_1in5_WIDTH-1, 95 , 0x08,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 95 , OLED_1in5_WIDTH-1, 105, 0x0A,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 105, OLED_1in5_WIDTH-1, 115, 0x0C,
DOT_PIXEL_1X1, DRAW_FILL_FULL);
Paint_DrawRectangle(0, 115, OLED_1in5_WIDTH-1, 125, 0x0F,

```

```
DOT_PIXEL_1X1, DRAW_FILL_FULL);  
    OLED_1in5_Display(BlackImage);  
  
    DEV_Module_Exit();  
  
    while(true) {}  
}
```

5. Створіть задачу, яка буде опитувати кнопку і записувати стан кнопки в чергу. Використайте `gpio_get()` для зчитування стану кнопки. Реалізуйте `debounce` (придушення брязкоту контактів). Передавайте значення `true/false` у чергу `xQueueButton`. Опитуйте кнопку кожні 50 мс.
6. Створіть задачі, які будуть зчитувати дані з потенціометра, фоторезистора та звукового сенсора і записувати у відповідні черги. Використайте ADC-канали `GPIO26–GPIO28`. Передавайте у черги значення 0–4095. Для кожного сенсора створіть окрему чергу (`xQueuePot`, `xQueueLight`, `xQueueSound`). Опитуйте сенсори кожні 100 мс.
7. Створіть задачу, яка буде брати дані сенсорів з черг і виводити інформацію на дисплей у вигляді прогрес-бару. Відображайте значення у вигляді шкали (напр., 0–100%). Оновлюйте екран з певною періодичністю (200–300 мс).
8. Додайте функцію вмикання/вимикання OLED-дисплея через кнопку. Якщо кнопка натиснута – перемикає стан змінної `display_on`. Якщо `display_on == false`, дисплей вимикається (очищується або переходить у `sleep`-режим).